

1. O que são templating engines e qual sua utilidade?

As **templating engines** são ferramentas utilizadas para gerar páginas HTML dinâmicas no servidor. Elas permitem combinar código HTML com dados vindos do backend, como informações de um banco de dados ou de uma API, tornando possível criar páginas personalizadas para cada usuário.

Sua principal utilidade é evitar a repetição de código HTML, facilitar a reutilização de layouts e permitir que os dados sejam inseridos dinamicamente antes da página ser enviada ao navegador.

Alguns exemplos de templating engines são:

- EJS (Embedded JavaScript)
 - Pug
 - Handlebars
-

2. Explique as diferenças entre EJS e Pug.

O **EJS** utiliza uma sintaxe muito semelhante ao HTML tradicional, permitindo inserir código JavaScript através de tags especiais como `<%= %>` e `<% %>`. Isso facilita o aprendizado para quem já conhece HTML.

Exemplo:

```
<h1>Olá, <%= nome %>!/</h1>
```

O **Pug** possui uma sintaxe baseada em indentação, dispensando diversas tags HTML. Isso torna o código mais enxuto, porém exige um período maior de adaptação.

Exemplo:

```
h1 Olá, #{nome}!
```

Principais diferenças:

EJS

Pug

Sintaxe semelhante ao HTML

Sintaxe baseada em indentação

Mais fácil para iniciantes

Código mais compacto

Utiliza JavaScript diretamente

Possui sintaxe própria

Ideal para quem conhece HTML

Ideal para projetos maiores e bem organizados

3. Como configurar o EJS em um projeto Express?

Primeiramente instala-se o EJS:

```
npm install ejs
```

Depois configura-se o Express:

```
const express = require("express");
```

```
const path = require("path");
```

```
const app = express();
```

```
app.set("view engine", "ejs");
```

```
app.set("views", path.join(__dirname, "views"));
```

Após essa configuração, o Express passa a procurar automaticamente os arquivos `.ejs` dentro da pasta `views`, utilizando-os para renderizar páginas HTML dinâmicas.

4. O que são partials e como eles auxiliam na reutilização de código?

Partials são pequenos arquivos reutilizáveis que representam partes comuns de uma página, como:

- cabeçalho;
- menu;
- rodapé;
- barra lateral.

Ao invés de copiar esses elementos para todas as páginas, basta incluí-los onde forem necessários.

Exemplo:

```
<%- include("partials/header") %>
```

Isso facilita a manutenção do projeto, pois qualquer alteração realizada no arquivo do cabeçalho será refletida automaticamente em todas as páginas que o utilizam.

5. Como os dados do MySQL podem ser renderizados em um template?

Primeiramente, o servidor realiza uma consulta ao banco de dados utilizando o MySQL.

Exemplo:

```
const [usuarios] = await pool.query("SELECT * FROM usuarios");
```

Depois, esses dados são enviados para o template:

```
res.render("usuarios", {  
  usuarios  
});
```

No arquivo EJS, é possível percorrer a lista utilizando JavaScript:

```
<% usuarios.forEach(usuario => { %>
```

```
<p><%= usuario.nome %></p>
```

```
<% }) %>
```

Dessa forma, os dados armazenados no banco aparecem automaticamente na página HTML.

6. Explique o papel do body-parser em aplicações Express.

O **body-parser** é responsável por interpretar os dados enviados pelo cliente em requisições HTTP.

Atualmente, suas funcionalidades já estão incorporadas ao Express através dos métodos:

```
app.use(express.json());
```

```
app.use(express.urlencoded({  
  extended: true  
}));
```

Esses métodos permitem que formulários HTML e requisições JSON sejam convertidos em objetos JavaScript acessíveis através de:

```
req.body
```

7. Quais são as boas práticas ao organizar templates em um projeto?

Algumas boas práticas são:

- manter todos os templates na pasta `views`;
- separar layouts em uma pasta específica;
- utilizar partials para componentes reutilizáveis;

- separar arquivos CSS e JavaScript na pasta `public`;
- utilizar nomes claros para os arquivos;
- evitar repetição de código HTML;
- organizar as rotas em arquivos separados.

Essa organização torna o projeto mais limpo, modular e fácil de manter.

8. Como funciona a inclusão de placeholders em layouts do EJS?

Os placeholders são áreas reservadas dentro de um layout onde o conteúdo específico de cada página será inserido.

Exemplo:

```
<html>
```

```
<head>
```

```
<title><%= title %></title>
```

```
</head>
```

```
<body>
```

```
<%- body %>
```

```
</body>
```

```
</html>
```

Ao renderizar uma página:

```
res.render("index", {  
  title: "Página Inicial"  
});
```

O valor "Página Inicial" será inserido automaticamente dentro da tag `<title>`.

Esse mecanismo permite reutilizar um único layout para diversas páginas diferentes.

9. Liste três aplicações práticas para o uso de formulários no Express.

Alguns exemplos são:

- cadastro de usuários;
- login de clientes;
- envio de mensagens pelo formulário de contato.

Outras aplicações incluem:

- cadastro de produtos;
 - atualização de informações;
 - envio de comentários;
 - pesquisas de satisfação.
-

10. Por que é importante validar dados recebidos de formulários?

A validação garante que os dados recebidos sejam corretos antes de serem armazenados no banco de dados.

Ela ajuda a evitar:

- campos vazios;
- e-mails inválidos;
- números incorretos;
- caracteres indesejados;
- erros na aplicação;
- vulnerabilidades de segurança.

Exemplo:

```
if (!nome || nome.trim().length < 2) {  
  
    return res.status(400).send("Nome inválido.");  
  
}
```

Além de aumentar a segurança da aplicação, a validação melhora a experiência do usuário ao informar claramente quando um dado foi preenchido de forma incorreta.